

Análise de Recursos, Timing, Desempenho, Gargalos, Limitações e Melhorias

Este texto resume os resultados da compilação Quartus Prime 25.1 (Lite), a interpretação de timing/desempenho e as principais limitações arquiteturais. Os números vêm de output_files/coprocessador.fit.summary, coprocessador.fit.rpt, coprocessador.sta.summary e coprocessador.sta.rpt.

1. Análise de recursos

1.1 Utilização global (Fitter)

Recurso	Usado / Disponível	%
Logic utilization (ALMs)	2.587 / 32.070	8 %
Registradores	1.615	—
Combinational ALUTs (lógica)	3.869	—
Block memory bits	229.376 / 4.065.280	6 %
RAM Blocks	28 / 397	7 %
DSP Blocks	24 / 87	28 %
PLLs	2 / 6	33 %
Pinos I/O	241 / 457	53 % *
LABs usados	401 / 3.207	13 %
Difficulty packing	Low	—

1.2 Distribuição aproximada por bloco

Módulo	Papel	Observação de custo
motor_sprites	32 sprites, prioridade, flip	Maior consumidor de ALMs (~1,2k)
banco_registradores	Atributos + double-buffer	Muitos registradores (~1,2k)
porta_estimulo	Cenários + mov_continuo	Lógica de sequenciamento (~300 ALMs)
rasterizador_multi	Polígonos (edge functions)	Principal usuário de DSP (24)
motor_background	Tilemap + scroll	Custo moderado (~27 ALMs)
vga_driver	Timing VGA + 2x2	Baixo (~43 ALMs)
compositor	Prioridade + transparência	Muito leve (~4 ALMs)
RAMs (tilemap/tiles/sprites)	altsyncram + .mif	Concentram block memory bits

Conclusão de área: o núcleo cabe com folga na 5CSEMA5F31C6. Há margem para interface MMIO, paleta programável e lógica de jogo nas etapas seguintes do PBL. O recurso relativamente mais pressionado é o DSP (28 %), ainda com sobra.

2. Análise de timing

2.1 Frequências e slacks reportados

Clock	Fmax (Slow 85 °C)	Setup slack	Hold
CLOCK_50	149,23 MHz	+13,3 ns	OK (+)
clk ~100 MHz (pllpara100)	147,73 MHz	+1,6 ns	OK (+)
clk_pix (meu_pll / VGA)	9,29 MHz	-94,5 ns (TNS alto)	OK (+)

Hold e minimum pulse width fecham em todos os clocks do relatório. O problema concentrado é setup no domínio clk_pix.

2.2 Interpretação das constraints

O Timing Analyzer registra create_clock apenas em CLOCK_50 (período 20 ns) e, em seguida, derive_clocks -period 1.0, com mensagem de que não havia base clocks de usuário suficientes para os outclk dos PLLs. Sem SDC explícito para clk_pix ≈ 25 MHz e clk_sys ≈ 100 MHz, os números de Fmax/slack do domínio VGA ficam distorcidos ou excessivamente pessimistas.

3. Desempenho funcional

3.1 Resolução e taxa

Resolução lógica 320×240 com expansão 2×2 para VGA 640×480. Pixel clock alvo ~25 MHz (≈60 Hz). Double-buffer de tilemap com troca alinhada ao VSYNC evita tearing na troca de buffers.

3.2 Throughput dos motores

- Background: pipeline de ~2 ciclos por pixel lógico; scroll H/V com wrap 320×240.
- Sprites: até 32 ativos; seleção por prioridade (maior attr_pri; empate = menor índice); latência de ROM/BRAM ~1 ciclo; flip H/V; transparência por índice de cor 0 no compositor.
- Polígonos: até 4 slots; retângulos e triângulos preenchidos por edge functions inteiras (sem ponto flutuante); saída essencialmente combinacional.
- Compositor: prioridade fixa sprite > polígono > background; cor 0 transparente (incluindo sprite sobre sprite, na medida em que o motor entrega um vencedor por pixel).

3.3 Interface de estímulo

A porta_estimulo substitui a futura MMIO com o mesmo contrato {wr_en, endereço[15:0], dado[8:0]}. Cenários SW[2:0] cobrem background+scroll, seleção/confirmação/espelho de sprites, movimento, polígonos, swap de buffer e estímulo inválido (LEDR). Displays 7 segmentos mostram S/P + índice.

4. Gargalos

4.1 Área / lógica

- motor_sprites é o maior consumidor de ALMs por causa dos 32 canais em paralelo. Escalar para mais de 32 sprites sem hierarquia de prioridade aumentaria o custo de forma quase linear.
- DSP: 24 blocos atribuídos majoritariamente ao rasterizador.

4.2 Timing / frequência

- Domínio clk_pix: caminho longo combinacional (sprites + raster + composição). É o gargalo de frequência do núcleo de vídeo.

4.3 Memória e banda

- Leituras de tilemap, padrões de tile e sprites competem no domínio do pixel. A resolução lógica 320×240 e a expansão 2×2 reduzem a pressão de banda pela metade em relação a 640×480 nativo — decisão que alivia esse gargalo.
- RAMs single-port limitam o número de amostragens simultâneas por ciclo; o desenho atual assume um vencedor de sprite por pixel, o que simplifica transparência entre sprites em cascata profunda.

4.4 Controle de demonstração

- porta_estimulo concentra sequências longas e tabelas de polígonos; útil para demonstração, mas não é o caminho de desempenho do jogo final (será substituída por MMIO).

5. Limitações

- Interface de comandos ainda é porta de estímulo (SW/KEY), não MMIO 32 bits mapeada no HPS/Linux — preparada pelo contrato de registradores, porém não integrada.
- Máximo de 32 sprites e 4 polígonos simultâneos; prioridade de camada fixa (três níveis) e entre sprites por atributo.
- Transparência binária (cor 0); sem alpha blending multi-nível.
- Timing de clk_pix não fechado de forma confiável no relatório publicado (constraints incompletas / caminho longo).
- Avisos de constant overflow em mov_continuo.v na síntese.

6. Melhorias possíveis

6.1 Pipeline e timing

- Registrar a saída do rasterizador e/ou do compositor (mais um estágio) para encurtar o caminho combinacional em clk_pix.
- Dividir a seleção de 32 sprites em duas etapas de 16 (árvore de prioridade pipelineada) se Fmax de clk_pix permanecer abaixo de 25 MHz após o SDC.
- Declarar set_clock_groups / false_paths de CDC onde o protocolo já garante estabilidade.

6.2 Funcionalidade gráfica

- Reinsere estágio de paleta (256×RRRGGBB) entre compositor e vga_driver, com registradores de carga via banco.
- Expandir slots de polígono ou adicionar modos (círculo aproximado, linhas) se o orçamento de DSP permitir.
- Suporte a mais padrões de sprite/tile via páginas de memória ou DMA a partir da SDRAM.

6.3 Integração de sistema

- Substituir porta_estimulo por interface MMIO 32 bits exposta ao HPS, mantendo o mapa de endereços do banco_registradores.
- Driver Linux + biblioteca C para o jogo, usando o mesmo contrato {endereço, dado} já exercitado na demonstração.
- Opcional: framebuffer em SDRAM para camadas estáticas ou UI, combinado com o pipeline tile/sprite atual.